Unveiling the Depth-Performance Dilemma in Split-Federated Fine-tuning of LLMs

Hariharan Ramesh [1] **Someshwaran Murugaiyan** [2] **Jyotikrishna Dass** [1]

[1]School of Electrical, Computing & Software Engineering, University of Arizona, Tucson, AZ, USA
[2]Vellore Institute of Technology, Vellore, India

hariharanr@arizona.edu someshwaran.2022@vitstudent.ac.in jdass@arizona.edu

Abstract

Split Federated Fine-tuning (SFF) is a promising paradigm for scaling Large Language Models (LLMs) by partitioning model depth between resource-constrained clients and a centralized server. While system incentives for throughput and privacy favor deep partitions, the impact of such configurations on model utility remains poorly understood. In this work, we identify and characterize the Depth-Performance Dilemma: the regime that maximizes system efficiency is precisely where fine-tuning quality collapses. Through a comprehensive audit across four model scales (GPT-2 to Llama-3-8B) and diverse benchmarks, we demonstrate that deeper partitions provide monotonic gains in throughput and privacy at the cost of catastrophic performance plateaus. We evaluate a suite of state-of-the-art federated adapter aggregation methods including AVG, STACK, SVD, and FREEZE, revealing that while these techniques are effective in standard Federated Learning, they fail to mitigate the artifacts unique to split architectures. Finally, we provide a mechanistic diagnosis for this failure, tracing the collapse to the near-isometric topology of Transformers, which allows aggregation noise to propagate without attenuation until it triggers Attention Collapse in the server partition. Our findings challenge the prevailing assumption that partition depth is a utility-neutral tuning knob and provide a structural foundation for stable distributed LLM fine-tuning.

1 Introduction

Large Language Models (LLMs) face a fundamental deployment challenge: privacy-sensitive environments necessitate Federated Learning (FL) [1], yet on-device fine-tuning remains computationally prohibitive even with Parameter-Efficient Fine-Tuning (PEFT) like LoRA [2]. Furthermore, fully decentralized training often suffers from the statistical instability of non-IID client drift. Split-Federated Fine-tuning (SFF) [3] resolves this by partitioning the model at a cut layer: clients execute shallow layers in parallel while a shared Main Server sequentially processes the remaining deep blocks (illustrated in Figure 2). By synchronizing client-side adapters via a Federated Server, SFF enables memory-feasible, privacy-aware optimization across distributed datasets.

Historically, split architectures were validated on Convolutional Neural Networks (CNNs), where pooling layers create an information "funnel" [4] that attenuates representations at depth. This property makes partition depth a convenient efficiency knob in CNN-based SFL, with manageable performance trade-offs. Recent LLM frameworks like SplitLoRA [5] and HSplitLoRA [6] implicitly extrapolate this intuition, assuming deep cuts remain "safe" for Transformers. However, Transformers maintain uniform dimensionality across all layers, lacking the spatial downsampling that stabilizes deep partitions in CNNs. Despite this structural divergence, the relationship between cut-layer depth and SFF performance for LLMs remains systematically uncharacterized.

Preprint.

In this work, we conduct a comprehensive empirical evaluation of SFF across diverse model scales and benchmarks. Through exhaustive cut-layer sweeps and multiple aggregation strategies, we identify a central **Depth-Performance Dilemma** (Figure 1): *while deepening the cut layer increases system throughput and reduces privacy leakage, model performance degrades consistently across every tested configuration.* Figure 1 makes this tension concrete for GPT-2 Small: as the cut layer ℓ_c deepens from early to deep cuts, system throughput rises steeply and privacy leakage decays toward zero (right axis), while task performance (NIST, left axis) falls consistently in the opposite direction. Because these two system-favorable trends move against the utility curve, the operating regime most attractive for system efficiency and privacy is precisely where fine-tuning quality collapses. We trace this collapse to a mismatch between federated aggregation and Transformer topology. While shallow cuts allow the server to absorb aggregation noise, deep cuts propagate noise through the residual architecture into "collapsing" attention layers in the server tail. This establishes the dilemma as intrinsic property of split-LLM under heterogeneity rather than a mere optimization failure.

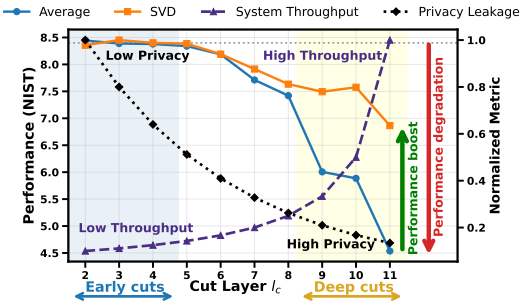


Figure 1: **Depth-Performance Dilemma.** As partition depth (cut layer ℓ_c) increases, system throughput rises and privacy leakage decays, yet task performance (NIST, left axis) degrades consistently. Throughput and normalized privacy leakage (right axis) are reported alongside performance for GPT-2 Small under heterogeneous clients. The throughput curve corresponds to the server-bound regime, where the throughput-optimal depth $d^\star{=}K/(K+\rho_s/\rho_c)$ meets or exceeds the deepest cut so the server is the bottleneck at every ℓ_c; see Appendix C.

Contributions. In summary, our contributions are as follows:

1. *Characterization of the Depth-Performance Dilemma:* We identify a fundamental tension in SFF where partition depths that maximize throughput and privacy consistently collapse fine-tuning quality. We show this trade-off holds consistently across four model scales (GPT-2 Small/Med/Large, Llama-3-8B) and three diverse benchmarks (E2E NLG, GLUE, GSM8K).
2. *Systematic Aggregation Audit:* We provide the first formal evaluation of AVERAGE [7], FREEZE [8], STACK [9], and SVD [10] in the split-federated setting. We characterize how their unique noise profiles interact with partition depth and client heterogeneity across exhaustive cut-layer sweeps.
3. *Mechanistic Diagnosis:* Using perturbation propagation and effective rank analysis, we trace the collapse to three compounding structural factors: aggregation noise, near-isometric noise propagation through the Transformer's residual architecture, and attention collapse in deep server layers that eliminates the nonlinear capacity required for error correction.

2 Related Work and Gaps

We position our work at the intersection of Split Learning (SL), LLM fine-tuning, and Federated aggregation theory, identifying a gap in noise-resilient aggregation for deep-cut Transformer topologies. Table 1 summarizes how SFF differs from existing paradigms.

Split Federated Learning (SFL). SL [15] decouples model depth from client constraints via a cut layer. Originally validated on CNNs, SL relies on spatial pooling to reduce bandwidth and filter noise. SFL extensions (SFL-V1/V2) [3] optimize synchronization but retain the structural premise that partition depth is a tunable knob for efficiency, ostensibly independent of model topology. While CNN-based audits [12] and theoretical analyses [11] suggest depth-invariance, these insights are coupled to CNN-specific inductive biases (spatial downsampling) that are absent in Transformers.

Split LLMs. Recent adaptations of SL to Transformers prioritize system feasibility over topological (structural) stability. Enabling frameworks like SplitLoRA [5] and HSplitLoRA [6] implement distributed fine-tuning, while optimizations like SplitFrozen [13] and SplitLLM [16] focus on reducing device load or inference latency. Privacy-centric works [17, 8] treat the cut layer strictly as a security boundary. Crucially, these studies tacitly treat partition depth as utility-neutral, ignoring how the constant-width near-isometric structure of a Transformer propagates aggregation artifacts.

Table 1: **The SFL Landscape.** Comparison of Split Learning approaches. While CNN-based works have extensively analyzed role of cut layer depth, LLM-based works have largely treated the cut layer as a static efficiency knob or privacy boundary. **Ours is the first to systematically audit the impact of deep partitions in Transformers performance across various aggregation strategies**. (D denotes cut layer depth and P denotes performance; arrows indicate trend direction trends as depth increases.)

WORKS	MODEL	AGGREGATION	DEPTH ANALYSIS	LORA RANK HETERO.
SFL [3]	CNNS	✓ (AVG)	✗	N/A
ANALYSIS [11]	CNNS	✓ (AVG)	$D \uparrow P \uparrow$	N/A
ANALYSIS [12]	CNNS	✓ (AVG)	$D \uparrow P \downarrow$	N/A
SPLITLORA [5]	LLMS	✓ (AVG)	✗	✗
SPLITFROZEN [13]	LLMS	✓ (AVG)	✗	✗
FSL-SAGE [14]	CNN/LLM	✓ (AVG)	✗	✗
HSPLITLORA [6]	LLMS	✓ (STACK)	✗	✓
DEPTH-PERFORMANCE DILEMMA (OURS)	**LLMS**	**AVERAGE, FREEZE, STACK, SVD**	$D \uparrow P \downarrow$	✓

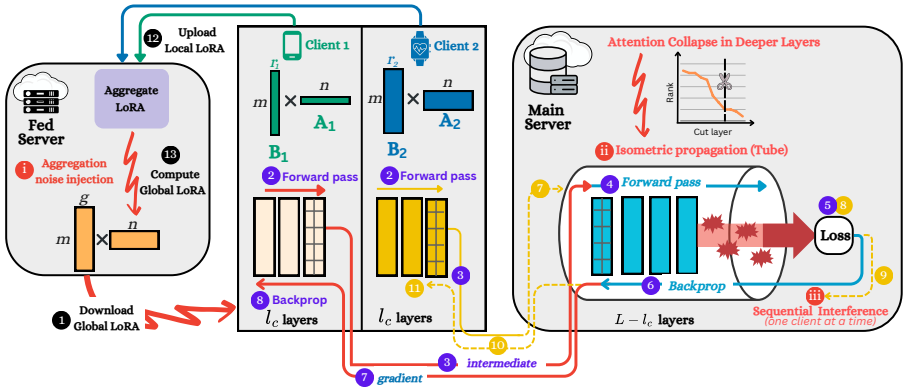


Figure 2: **Split-Federated Fine-tuning of LLMs.** Clients parallelize the shallow partition while the Main Server sequentially processes the shared deep blocks. Fine-tuning performance degrades consistently with cut-layer depth due to: (i) aggregation noise from heterogeneous LoRA updates; (ii) near-isometric propagation where residual connections transmit noise unattenuated; and (iii) attention collapse where deep server layers lack the rank and nonlinear capacity to correct errors.

Federated Adapter Aggregation. Aggregating LoRA adapters presents a unique challenge: standard averaging (AVG) is mathematically ill-suited for factorized matrices ($\mathbf{W} + \mathbf{BA}$) due to the non-linearity of the product: $\mathbb{E}[\mathbf{B}]\mathbb{E}[\mathbf{A}] \neq \mathbb{E}[\mathbf{BA}]$ [7, 18]. While FL-centric solutions like STACK [9], FREEZE [19], and SVD [10, 20] aim to restore mathematical accuracy, reduce communication overhead, or support heterogeneous ranks, they have been evaluated exclusively in standard Federated setups. Consequently, their impact on Split Federated Finetuning remains unexamined, particularly in the context of a shared server partition where aggregation artifacts directly influence the sequential processing of deep blocks.

Gaps. We summarize two main gaps as follows: (i) *Topological Sensitivity:* determining whether increasing the split depth for system efficiency creates a tipping point that disrupts the LLM's performance, and (ii) *LoRA Aggregation Impact:* While methods exist to combine client updates under federated setup, they have not been evaluated and compared in a split-model context with a shared server partition with sequential updates.

3 The Depth-Performance Dilemma

To address the above gaps, we begin by establishing the operational case for deep partitioning in SFF (Section 3.1), then demonstrate empirically that the very depths favored by system incentives are precisely where fine-tuning quality collapses (Section 3.2). This tension, shown in Figure 1, which we term the **Depth-Performance Dilemma**, motivates the diagnostic investigation in Section 4.

3.1 System Benefits of Deep Cuts

An overview of the SFF pipeline is shown in Figure 2; full pseudocode is in Appendix B. We consider N clients $\mathcal{C} = \{1, \ldots, N\}$, each holding a private dataset $\mathcal{D}_i$. The model is partitioned at cut layer ℓ_c: client i executes the shallow sub-model $f_C(\cdot; \Theta_{C,i})$ through the first ℓ_c Transformer blocks, while a shared *Main Server* executes the remaining deep sub-model $f_S(\cdot; \Theta_S)$. Clients transmit cut-layer activations to the server and receive activation gradients in return, following the parallel execution model of SplitLoRA [5]. A separate *Federated Server* periodically aggregates client-side LoRA [2] adapters and broadcasts a global adapter back to all clients.

Impact on Throughput. The primary constraint in SFF is the *sequential server bottleneck*: the Main Server processes client activations one at a time, so server-side compute directly throttles throughput. Unlike CNNs, where pooling layers create irregular compute profiles across depth, Transformer blocks are *uniform* in hidden dimension and cost, every layer shifted to the client yields a precise, predictable reduction in server load. Formally (Appendix C), in the server-bound regime $d < d^\star{=}K/(K + \rho_s/\rho_c)$, throughput scales as $\mathcal{T}(d) = \Theta\left(\frac{1}{1-d}\right)$ with $d = \ell_c/L$, so moving from $d{=}0.5$ to $d{=}0.9$ delivers up to a $5\times$ gain. For realistic client counts and compute ratios, $d^\star$ can reach the deepest cut (e.g., $d^\star{=}0.91$ at $K{=}10$, $\rho_s/\rho_c{=}1$; Table 7), keeping deep cuts throughput-favorable; where it does not, the deep-cut incentive is carried by privacy, which improves with depth independent of K and compute.

Impact on Privacy. The cut-layer representation $\mathbf{h}_{\ell_c}$ is the only signal the server ever observes about a client's private input. At shallow cuts these representations are close to raw token embeddings and can be inverted with high fidelity; at deep cuts, multi-head attention has progressively diffused token identity into the global context.

We validate this directly by training adversarial MLP inversion models that reconstruct input tokens from cut-layer hidden states (setup in Appendix G). We define *Empirical Privacy* $\text{EP} = (1{-}\text{RR}){\times}100$, where RR is token recovery rate of the strongest attacker. Figure 3 shows EP across GPT-2 Small, Medium, and Large. At shallow cuts ($d < 0.4$), EP remains near zero, tokens are recoverable with high accuracy from the cut-layer hidden states. Beyond $d \approx 0.8$, EP rises sharply and reaches 100 at deep cuts ($d > 0.9$), indicating that representations at this depth carry negligible recoverable information about the original input, consistent across all three GPT-2 scales and, at 8B scale, for Llama-3-8B-Instruct (Appendix G, Figure 9).

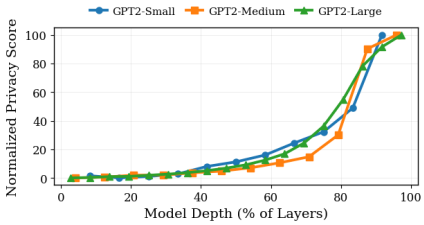


Figure 3: **Normalized Privacy Score vs. partition depth.** Score $= (1{-}\text{RR}){\times}100$, where RR is token recovery rate of an adversarial MLP inversion model. Score rises sharply beyond $\approx$80% depth and reaches 100 at deep cuts.

Key Takeaway: Deeper partitions maximize both throughput and privacy. These dual incentives position deep partitioning as the unambiguous system-optimal regime.

3.2 Impact of Depth on Fine-tuning Performance

Experimental setup. We evaluate SFF across four models, three benchmarks, and up to 100 clients, making this the most extensive empirical study of cut-layer depth in split-federated LLM fine-tuning to date. We evaluate SFF across GPT-2 Small/Medium/Large [21] on E2E NLG [22] and Llama-3-8B-Instruct [23] on GSM8K [24], with cut layers swept exhaustively ($\ell_c \in \{1, \ldots, L{-}1\}$ for GPT-2; stride-3 sweep for LLaMA; 30-client federated setup with 3 sampled per round for 50 communication rounds). All experiments use non-IID data (Dirichlet, $\alpha{=}0.5$), with both heterogeneous ($r_i \in \{4, 8, 16\}$) and homogeneous ($r{=}16$) LoRA rank configurations. Generalization to natural language understanding is validated on GLUE [25] (GPT-2 Small, 100 clients); results are in Appendix A.2. Performance is measured by PPL and task metrics (BLEU, NIST, METEOR, ROUGE-L, CIDEr for E2E; accuracy for GSM8K and GLUE). Full hyperparameter details and complexity comparisons are in Appendix A and Appendix F.

Aggregation protocols. Aggregation design in SFF remains largely underexplored. To date, only two strategies have been studied in prior SFL works: SplitLoRA [5], which applies simple averaging, and HSplitLoRA [6], which uses stacking to handle heterogeneous ranks. Beyond these, we conduct the *first systematic audit* of representative federated LoRA aggregation methods in the split setting, a contribution in itself, since these methods were designed and evaluated exclusively for standard (non-split) FL. We apply LoRA to the query and value projection matrices of each attention block, parameterizing updates as $\Delta\mathbf{W} = \mathbf{BA}$ where $\mathbf{B} \in \mathbb{R}^{d\times r}$, $\mathbf{A} \in \mathbb{R}^{r\times d}$. Every $I = 100$ local steps, client adapters are synchronized via one of four strategies: **(i) AVERAGE [7]** averages adapters element-wise, introducing cross-term interference ($\mathbf{B}_{\text{agg}}\mathbf{A}_{\text{agg}} \neq \sum_i \alpha_i \mathbf{B}_i\mathbf{A}_i$). **(ii) FREEZE [19]** fixes a shared projection $\mathbf{A}_{\text{fixed}}$ and trains only $\mathbf{B}$, eliminating cross-term noise at the cost of halved adaptation capacity. **(iii) STACK [9]** concatenates adapters, preserving all client subspaces exactly but scaling the global rank as $N \times r$. **(iv) SVD [10]** (following FlexLoRA) aggregates in update space, $\Delta\mathbf{W}_{\text{agg}} = \sum_i \alpha_i \mathbf{B}_i\mathbf{A}_i$, applies SVD, and delivers client-specific global adapters $\mathbf{B}_k^{(g)} = \mathbf{U}_{[:,:r_k]}\mathbf{\Sigma}_{[:r_k,:r_k]}$, $\mathbf{A}_k^{(g)} = \mathbf{V}^\top_{[:r_k,:]}$, truncated to each client's rank r_k. Methods requiring aligned factors apply rank alignment via zero-padding following HetLoRA [18].

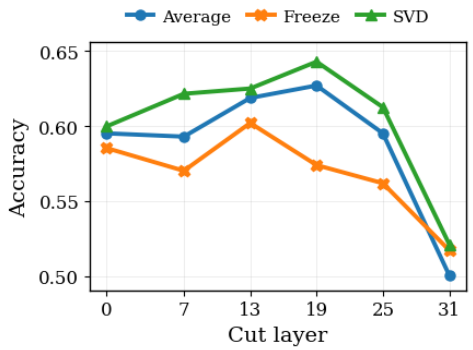


Figure 4: **GSM8K accuracy vs. cut-layer depth** for Llama-3-8B-Instruct.

Depth degrades performance across all aggregation methods. Figure 5 reports generation metrics across cut layers for GPT-2 Small and Medium; Figure 7 reports validation PPL for all three GPT-2 scales; and Figure 4 reports GSM8K accuracy for Llama-3-8B-Instruct. Across every model, benchmark, and aggregation method, performance at deep cuts is substantially lower than at shallow cuts. GSM8K accuracy plateaus at shallow-to-moderate cuts before declining at deep cuts, consistently across all aggregation methods. Across both GPT-2 and LLaMA, the deep-cut regime, which maximizes throughput and privacy, consistently incurs the largest performance penalty. The degradation is not uniform across methods: FREEZE collapses most sharply at deep cuts due to its constrained adaptation subspace; AVERAGE degrades steadily; and STACK and SVD are more robust, though neither is immune. The key observation is that *no aggregation strategy escapes depth-induced degradation*, the dilemma is not an artifact of any single method's design. The rate and severity of collapse does, however, depend on the aggregation rule, a distinction we pursue in Section 4.

Figure 6 confirms this is not a convergence failure: training loss converges in every configuration, but deeper cuts converge to worse optima. The shallow–deep separation in PPL emerges within the first few communication rounds and widens throughout training.

Key Takeaway: Section 3.1 demonstrates that system throughput and data privacy both improve with deeper partitioning. Conversely, Section 3.2 reveals that fine-tuning performance degrades consistently across depth. This establishes the **Depth-Performance Dilemma**: the system-optimal operating regime is precisely where model quality collapses. Section 4 provides a mechanistic investigation into the structural causes of this failure.

4 Diagnosing the Depth-Performance Dilemma

The experiments in Section 3 establish that deep cuts consistently produce the worst performance across all models and aggregation methods, and that the severity of collapse varies by aggregation strategy. We investigate three research questions, each building naturally on the previous:

[RQ1]. ***Is performance collapse driven by aggregation method, or is it intrinsic to partition depth?*** In Section 4.1, we compare all four strategies under identical conditions. We observe two distinct regimes: at shallow cuts, aggregation choice has negligible impact; at deep cuts, methods diverge substantially and consistently across all models and tasks. These findings demonstrate that the specific mechanism utilized to combine adapters assumes critical importance at greater depths, raising the fundamental question of what constitutes this operational dynamic.

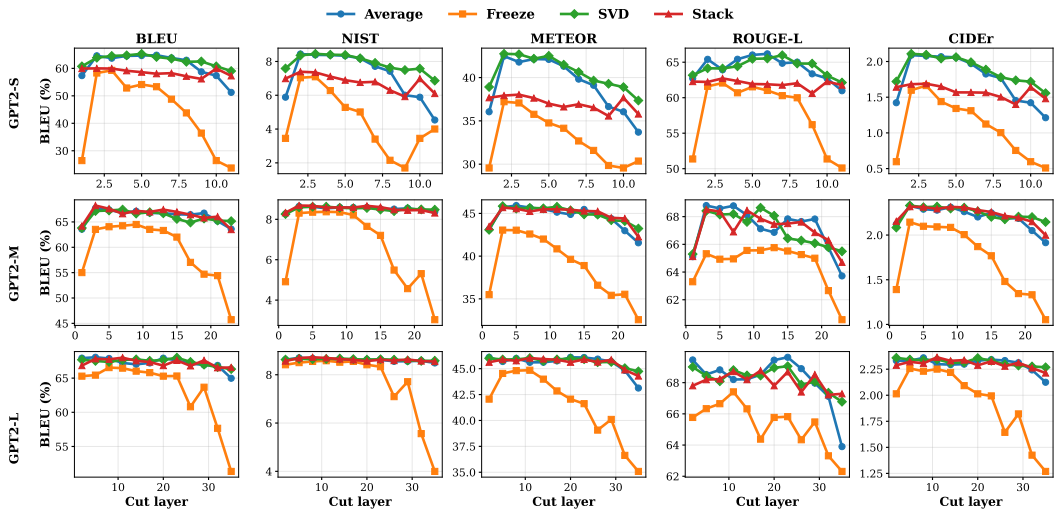
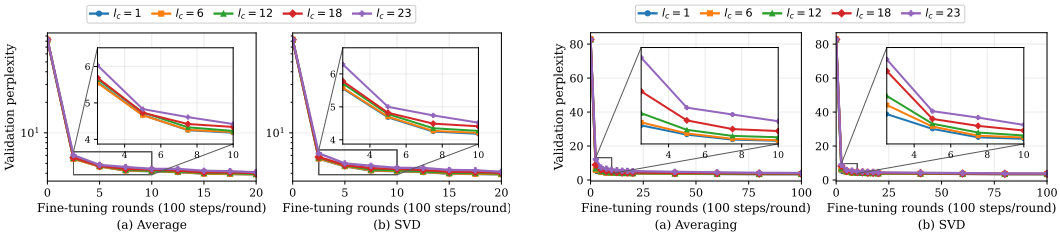


Figure 5: **Generation quality across cut-layer depth and aggregation method.** GPT-2 Small (top) and Medium (bottom) performance across cut layers for five metrics (BLEU, NIST, METEOR, ROUGE-L, CIDEr; BLEU/METEOR/ROUGE-L in %). Across all methods, performance at deep cuts is substantially lower than at shallow cuts; the rate of collapse varies by method but no method is immune. The performance cliff aligns with the attention collapse thresholds identified in Figure 8.



(a) GPT-2 M - Homogeneous (r=8)

(b) GPT-2 M - Heterogeneous

Figure 6: **Convergence under increasing cut-layer depth.** Validation perplexity (PPL; lower is better) across SFF rounds (100 steps/round) for GPT-2 Medium under (a) homogeneous and (b) heterogeneous LoRA rank configurations. Main panels use log-y scale; insets zoom early training with linear-y. Deeper partitions exhibit slower convergence and higher plateaued PPL. This degradation is amplified under heterogeneity, identifying federated aggregation as the primary failure mode.

This establishes that something about *how adapters are combined* becomes critically important at depth, and naturally asks what that something is.

[RQ2]. ***Why do aggregation methods diverge at deep cuts but not shallow ones?***
Section 4.2 answers this by characterizing the noise each method produces when combining heterogeneous adapters, then showing that the Transformer server, unlike a CNN, propagates this noise intact from cut point to output rather than attenuating it. Shallow cuts tolerate any noise level because the long server partition provides a buffer; deep cuts do not, because the buffer has nearly vanished.

[RQ3]. ***Why does the server lack the capacity to correct incoming noise at depth?***
Section 4.3 shows that deep server layers degenerate into near-identity mappings, a phenomenon we observe empirically via effective rank analysis, eliminating the nonlinear processing capacity that would otherwise provide implicit error correction. Sequential server updates compound this by forcing client gradients into a low-dimensional manifold where they inevitably conflict.

4.1 Impact of Aggregation on Depth-Induced Degradation

To address **[RQ1]**, we isolate the aggregation method as the sole variable: model, dataset, client count, and cut layer are held fixed across conditions. Server topology and sequential update dynamics

are therefore identical; any performance divergence must be attributed to how adapters are combined at the Federated Server.

Figure 7 and Table 3 report results across all cut-layer depths for GPT-2 Small, Medium, and Large, with a Federated Learning (FL) baseline included for reference, representing the case where the entire model is aggregated at every round with no server partition.

Shallow cuts: aggregation-invariant regime. At shallow cuts, aggregation method has little impact on final performance. For GPT-2 Small, AVERAGE, STACK, and SVD cluster tightly with AVG5 between 0.564 and 0.617; similar groupings appear for Medium (0.661–0.665) and Large (0.675–0.680). When the server retains many layers, the residual depth is sufficient to absorb whatever the Federated Server delivers, regardless of how adapters were combined.

Deep cuts: aggregation-sensitive regime. At depth, the picture changes decisively. Performance diverges substantially across methods and the divergence grows with ℓ_c. For GPT-2 Small: SVD achieves AVG5 $= 0.570$, STACK $= 0.528$, AVERAGE $= 0.498$, FREEZE $= 0.310$. GPT-2 Medium and Large follow the same ordering, as does Llama-3-8B-Instruct on GSM8K (Table 2). *If the choice of aggregation method were irrelevant at depth, all methods would degrade identically, they do not.* The systematic ordering across all models and tasks confirms that aggregation is a controllable driver of the collapse. What makes one method degrade less than another is the focus of Section 4.2.

Note that every method still degrades with depth to some degree, even SVD drops from AVG5 $= 0.617$ (shallow) to 0.570 (deep) on GPT-2 Small. This residual gap, present regardless of aggregation choice, points to a structural property of the Transformer server that no aggregation method can address, examined in Section 4.3.

Table 2: **Llama-3-8B GSM8K accuracy.** Shallow and deep cuts correspond to $l_c = 1$ and $l_c = 31$; FedB is a standard Federated Learning baseline with no server partition.

Method	Shallow	Deep	FedB
AVERAGE	0.595	0.500	0.420
FREEZE	0.585	0.517	0.420
SVD	**0.600**	**0.521**	**0.428**

FL baseline: the limiting case. Strikingly, *every SFF configuration outperforms standard FL*, including deep cuts. In FL, the entire model is subject to aggregation noise at every round with no server partition to provide any downstream processing. In SFF, even the deepest cut retains at least one server layer between the noise injection point and the prediction head. The FL baseline thus represents the true lower bound of the aggregation-noise regime, and SFF's advantage over it narrows as the cut deepens and the server partition shrinks toward zero. This pattern holds at 8B scale: on GSM8K, the deep-cut accuracies of 0.500, 0.517, and 0.521 for AVERAGE, FREEZE, and SVD all exceed the corresponding FL baseline of 0.420, 0.420, and 0.428 (Table 2).

Key Takeaway: Aggregation choice is irrelevant at shallow cuts but decisive at deep cuts, where methods diverge substantially and consistently across all models and tasks. Every SFF configuration outperforms standard FL, confirming that even a thin server partition provides meaningful buffering. The systematic ordering across methods raises the question of what distinguishes them, which we answer in Section 4.2.

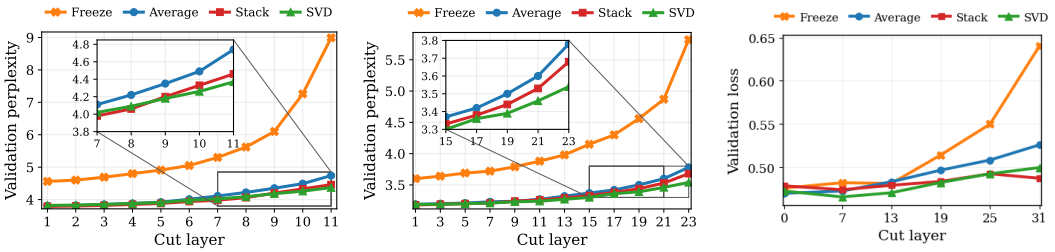
4.2 Aggregation Methods Diverge at Depth

Section 4.1 establishes that aggregation choice becomes decisive at deep cuts. Addressing **[RQ2]**: what distinguishes the methods? Each makes a different structural trade-off when combining heterogeneous low-rank adapters, producing qualitatively different noise characterized formally in Appendix D. AVERAGE flattens the singular value spectrum: the low-rank task signal contributes $O(1/K)$ to the aggregate while cross-client noise contributes $O(1/\sqrt{K})$, a ratio that worsens with client count, yielding a high-rank "white noise" update that dilutes task-specific directions. FREEZE avoids cross-term interference by fixing a shared projection, but constrains all clients to a single subspace, creating an irreducible approximation bias for any task features orthogonal to it. STACK preserves all client subspaces exactly, but interference noise also grows as $O(1/\sqrt{K})$. SVD [10] aggregates in update space and retains only the dominant singular components, truncating to each client's local rank r_k to project out the high-frequency noise tail while preserving the principal directions of cross-client consensus, explaining why it degrades least at deep cuts.

Table 3: **GPT-2 performance at shallow and deep cuts, with FL baseline.** Shallow (resp. deep) values average over the first (resp. last) 25% of cut positions. FedB is a standard Federated Learning baseline where the full model is aggregated at every round with no server partition. Metrics: B=BLEU (%), N=NIST, M=METEOR (%), R=ROUGE-L (%), C=CIDEr; AVG5 is the geometric mean of per-metric normalized scores. All SFF configurations outperform FL; performance at deep cuts is lower than at shallow cuts for every method, but the margin over FL narrows as the cut deepens. **Bold**: best; underline: second-best per model, depth, and metric.

Model	Method	Shallow cut						Deep cut						FedB
		B	N	M	R	C	AVG5	B	N	M	R	C	AVG5	AVG5
GPT2-S	Average	61.88	7.57	40.10	**64.02**	1.86	0.595	55.79	5.48	35.47	62.35	1.36	0.498	0.435
	Freeze	48.00	5.86	34.61	58.35	1.29	0.476	28.86	3.05	29.92	52.56	0.62	0.310	0.220
	Stack	59.95	7.24	37.87	62.40	1.67	0.564	57.79	6.34	36.34	61.55	1.51	0.528	**0.568**
	SVD	**63.05**	**8.13**	**41.45**	63.81	**1.97**	**0.617**	**60.77**	**7.31**	**38.51**	**63.35**	**1.67**	**0.570**	0.557
GPT2-M	Average	66.21	8.51	**44.92**	**67.50**	2.25	0.663	65.17	8.46	42.98	65.78	2.05	0.639	0.511
	Freeze	60.86	7.19	40.53	64.52	1.88	0.590	51.61	4.30	34.51	62.74	1.24	0.457	0.196
	Stack	**66.67**	**8.56**	44.90	67.33	**2.26**	**0.665**	65.11	8.41	43.72	**65.95**	2.11	0.645	**0.624**
	SVD	66.07	8.49	44.83	67.29	2.24	0.661	**65.37**	**8.50**	**43.85**	65.79	**2.18**	**0.651**	0.622
GPT2-L	Average	**67.95**	**8.67**	45.93	**68.93**	2.34	**0.680**	66.26	8.54	44.55	66.33	2.23	0.659	0.603
	Freeze	65.76	8.49	43.80	66.25	2.17	0.651	57.56	5.76	37.27	63.70	1.51	0.524	0.402
	Stack	67.47	8.66	45.75	68.07	2.31	0.675	**66.89**	8.58	45.01	**67.67**	2.26	0.667	0.642
	SVD	67.52	8.64	**45.93**	68.51	**2.35**	0.678	66.64	**8.58**	**45.15**	67.40	**2.28**	**0.667**	**0.648**

Amongst SFF shallow cut vs SFF deep cut vs FedB (≡ SFF with cut layer as last layer ≡ all layers on client): Best Worst



(a) GPT-2 Small (b) GPT-2 Medium (c) Llama-3-8B (GSM8K)

Figure 7: **Validation loss vs. cut-layer depth.** Loss (lower is better) as a function of ℓ_c for GPT-2 Small, Medium (heterogeneous LoRA ranks), and Llama-3-8B-Instruct on GSM8K (homogeneous, r=16). At shallow cuts, all methods perform similarly. At deep cuts, performance diverges sharply by aggregation method across all models and task types, with FREEZE collapsing most severely and SVD retaining the strongest performance.

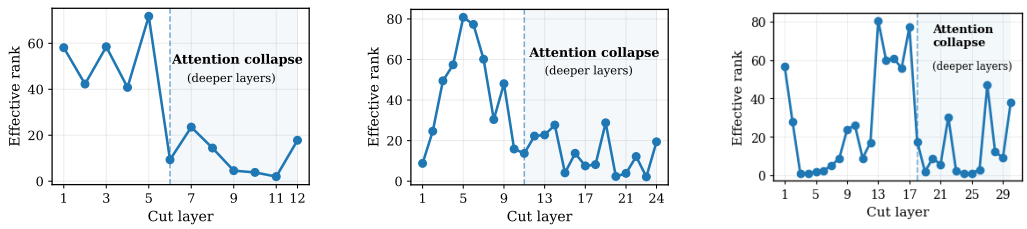
The reason these noise differences matter more at deep cuts than shallow ones is a structural property of the Transformer server. In CNN-based split architectures, pooling layers progressively compress spatial resolution, attenuating perturbations as they pass through successive layers. Transformers have no analogous mechanism: residual connections and layer normalization are designed to *preserve* signal norms, not reduce them. This means that whatever noise enters the server partition at the cut point exits essentially intact, a claim we verify directly.

We verify this via a perturbation propagation experiment. For each model and cut layer ℓ_c, we inject an L2-normalized Gaussian noise vector ε at the cut-point hidden state and measure $\|\tilde{x}_L - x_L\|/\|\varepsilon\|$ at the final layer after running the perturbed forward pass through the remaining server layers on frozen pretrained weights (no LoRA, no training, no federation), averaged over 64 random inputs.

Table 4 shows that across all models and cut depths, the perturbation ratio never drops below 1.0. At $\ell_c = L-3$, GPT-2 Small yields 1.89±0.08; even

Table 4: **Noise propagation through the server partition.** Final-layer perturbation ratio for the three deepest cut layers per GPT-2 model (±std, 64 inputs). Values $\geq$ 1.0 confirm that noise injected at the cut point exits the server partition unattenuated. A contractive (CNN-like) architecture would produce values decaying toward 0.

GPT-2	$\ell_c = L-3$	$\ell_c = L-2$	$\ell_c = L-1$
S $\{7, 9, 11\}$	1.89±0.08	1.55±0.05	1.23±0.06
M $\{14, 18, 23\}$	1.58±0.06	1.37±0.02	1.13±0.01
L $\{21, 28, 35\}$	1.46±0.04	1.22±0.02	1.04±0.01



(a) GPT-2 S ($\ell \approx 6$) (b) GPT-2 M ($\ell \approx 11$) (c) Llama-3-8B ($\ell \approx 19$)

Figure 8: **Attention collapse across depth.** Maximum effective rank (90% spectral energy threshold) of attention matrices per layer for GPT-2 Small, Medium, and Llama-3-8B-Instruct. Rank drops sharply after $\ell \approx \{6, 11, 19\}$ respectively, coinciding with the depths at which performance degradation accelerates in Figures 7 and 4. The collapse threshold scales with model depth, consistently occurring at approximately 50–60% of total layer depth.

at $\ell_c = L-1$ it remains 1.23 ± 0.06. A contractive architecture would produce ratios decaying toward 0. Instead, aggregation noise injected at the cut point exits the server partition *unattenuated or amplified*, propagating through the residual stream without shrinking. This explains why aggregation noise that is absorbed harmlessly at shallow cuts (where many layers follow the cut point) becomes catastrophic at deep cuts (where the server partition is thin and provides no compression). The transition from aggregation-invariant to aggregation-sensitive behavior observed in Section 4.1 is therefore partly explained by the server's loss of buffering capacity as depth increases.

This result also clarifies the FL baseline finding: in standard FL, aggregation noise is injected across the *entire* model with no partition to provide even minimal buffering. Every layer processes noisy updates from the first forward pass, and there are no subsequent layers to compensate. SFF's advantage over FL, even at deep cuts, follows directly from the fact that at least some unperturbed server processing occurs before the prediction head, a buffer that vanishes only in the limit $\ell_c \to L$.

Key Takeaway: Unlike CNNs, the Transformer server partition does not attenuate aggregation noise, it propagates it intact from cut point to output. This near-isometric propagation explains why noise that is harmless at shallow cuts, where the server provides a long buffer, becomes catastrophic at deep cuts where the buffer is thin. It also explains why all SFF configurations outperform FL: even the deepest cut retains some unperturbed server processing.

4.3 Structural Failure in Deep Server Layers

Addressing **[RQ3]**: the propagation result shows that noise is not attenuated, but does not fully account for the sharp *acceleration* in performance degradation visible in Figure 7 beyond a specific depth. If propagation were uniformly near-isometric at all depths, degradation would scale smoothly. Instead, we observe a characteristic inflection that aligns with a discrete structural change in the server's processing capacity.

Attention Collapse. Following [26], we measure the *effective rank* of attention matrices at each layer by computing the singular value spectrum and applying a 90% spectral energy threshold. Figure 8 shows results for GPT-2 Small, Medium, and Llama-3-8B. Effective rank is high in early layers and drops sharply at a characteristic depth: $\ell \approx 6$ for GPT-2 Small, $\ell \approx 11$ for Medium, and $\ell \approx 19$ for LLaMA. Beyond this point, attention matrices degenerate into near-identity, low-rank mappings, **Attention Collapse**. These thresholds align precisely with the depths at which performance degradation accelerates in Figure 7 and Figure 4: for LLaMA, the accuracy decline across all aggregation methods begins at $\ell_c \approx 19$, exactly matching the measured collapse threshold. Prior to collapse, high-rank server layers can project noisy activations toward the data manifold; after collapse, the server tail consists of near-identity layers with no corrective capacity, and noise propagating through the residual stream exits directly into the prediction head.

Sequential server processing. Attention collapse compounds a second structural property of SFF: the Main Server processes client activations one at a time, so update steps for client k shift the shared server state before client j is processed. At shallow cuts, high-rank server layers provide geometric

freedom for diverse client gradients to occupy non-conflicting subspaces. At deep cuts, attention collapse reduces the server's effective dimensionality: when task diversity exceeds geometric capacity, sequential updates inevitably collide, and each client partially overwrites the progress of the others. The residual performance gap that persists under SVD, which addresses aggregation noise upstream but not server-side dynamics, is consistent with this compounding factor.

Key Takeaway: Deep server layers degenerate via Attention Collapse, eliminating the nonlinear capacity needed to correct incoming noise. This explains the characteristic performance cliff: before the collapse threshold, the server can partially compensate for aggregation noise; after it, noise arrives intact and exits uncorrected. The collapse threshold scales consistently with model depth, occurring at $\approx$50–60% of total layers across GPT-2 and LLaMA. Sequential server processing further narrows the geometric space available for client updates to coexist.

Our findings yield a coherent account of the Depth-Performance Dilemma. Aggregation noise is the dominant controllable factor: methods that preserve the low-rank update manifold (SVD, STACK) degrade substantially less at deep cuts, and all SFF configurations outperform standard FL by retaining at least some unperturbed server processing. Near-isometric propagation ensures that noise persists throughout the partition, while attention collapse removes the nonlinear capacity required to recover.

5 Conclusion and Future Work

We conducted the first systematic empirical investigation of cut-layer depth in Split-Federated Fine-tuning of LLMs, uncovering a fundamental **Depth-Performance Dilemma**: the partition depths that maximize throughput and privacy are precisely where fine-tuning quality collapses, a finding that holds across GPT-2 Small/Medium/Large and Llama-3-8B-Instruct, on E2E NLG, GSM8K, and GLUE, under every aggregation method studied. Through controlled experiments, we traced the collapse to three compounding structural factors: aggregation noise whose severity is governed by how well each method preserves the low-rank update manifold; near-isometric noise propagation through the Transformer's residual architecture, which delivers aggregation artifacts intact to the output; and attention collapse in deep server layers, which eliminates corrective capacity precisely where noise arrives most intact, further compounded by sequential server processing. These results challenge the assumption, implicit in all prior SFF work, that partition depth is a utility-neutral efficiency knob, depth interacts with Transformer topology and aggregation design in ways that determine whether fine-tuning succeeds or collapses. Two concrete directions follow from this diagnosis. First, server-side execution strategies that accumulate gradients across all clients before updating, approximating centralized optimization, would eliminate the moving-target problem and recover the geometric separation that attention collapse currently destroys. Second, the aggregation design space remains open: even in standard FL, federated LoRA methods suffer from subspace misalignment under non-IID data [27], and in SFF this misaligned noise enters a near-isometric partition immediately, amplifying its consequences. Aggregation schemes that explicitly account for subspace alignment and cut-layer topology represent a principled path toward resolving the Depth-Performance Dilemma.

References

[1] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Aguera y Arcas. Communication-Efficient Learning of Deep Networks from Decentralized Data. In Aarti Singh and Jerry Zhu, editors, *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, volume 54 of *Proceedings of Machine Learning Research*, pages 1273–1282. PMLR, 20–22 Apr 2017.

[2] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *The Tenth International Conference on Learning Representations, ICLR 2022*. OpenReview.net, 2022.

[3] Chandra Thapa, Pathum Chamikara Mahawaga Arachchige, Seyit Camtepe, and Lichao Sun. Splitfed: When federated learning meets split learning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 36(8):8485–8493, Jun. 2022.

[4] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. Imagenet classification with deep convolutional neural networks. *Commun. ACM*, 60(6):84–90, May 2017.

[5] Zheng Lin, Xuanjie Hu, Yuxin Zhang, Zhe Chen, Zihan Fang, Xianhao Chen, Ang Li, Praneeth Vepakomma, and Yue Gao. Splitlora: A split parameter-efficient fine-tuning framework for large language models, 2024.

[6] Zheng Lin, Yu xin Zhang, Zhe Chen, Zihan Fang, Xianhao Chen, Praneeth Vepakomma, Wei Ni, Jun Luo, and Yue Gao. Hsplitlora: A heterogeneous split parameter-efficient fine-tuning framework for large language models. *ArXiv*, abs/2505.02795, 2025.

[7] Jianyi Zhang, Saeed Vahidian, Martin Kuo, Chunyuan Li, Ruiyi Zhang, Tong Yu, Guoyin Wang, and Yiran Chen. Towards building the federatedGPT: Federated instruction tuning. In *International Workshop on Federated Learning in the Age of Foundation Models in Conjunction with NeurIPS 2023*, 2023.

[8] Yiming Wang, Yu Lin, Xiaodong Zeng, and Guannan Zhang. Privatelora for efficient privacy preserving llm, 2023.

[9] Yexiao He, Ang Li, Lingjuan Lyu, Zheyu Shen, Guoheng Sun, Hongyi Wang, and Ziyao Wang. Flora: Federated fine-tuning large language models with heterogeneous low-rank adaptations. pages 22513–22533, 01 2024.

[10] Jiamu Bai, Daoyuan Chen, Bingchen Qian, Liuyi Yao, and Yaliang Li. Federated fine-tuning of large language models under heterogeneous tasks and client resources. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.

[11] Pengchao Han, Chao Huang, Geng Tian, Ming Tang, and Xin Liu. Convergence analysis of split federated learning on heterogeneous data. In *Proceedings of the 38th International Conference on Neural Information Processing Systems*, NIPS '24, Red Hook, NY, USA, 2024. Curran Associates Inc.

[12] Justin Dachille, Chao Huang, and Xin Liu. The impact of cut layer selection in split federated learning, 2024.

[13] Jian Ma, Xinchen Lyu, Jun Jiang, Qimei Cui, Haipeng Yao, and Xiaofeng Tao. Splitfrozen: Split learning with device-side model frozen for fine-tuning llm on heterogeneous resource-constrained devices, 2025.

[14] Srijith Nair, Michael Lin, Peizhong Ju, Amirreza Talebi, Elizabeth Serena Bentley, and Jia Liu. FSL-SAGE: Accelerating federated split learning via smashed activation gradient estimation. In *Forty-second International Conference on Machine Learning*, 2025.

[15] Praneeth Vepakomma, Otkrist Gupta, Tristan Swedish, and Ramesh Raskar. Split learning for health: Distributed deep learning without sharing raw patient data. *CoRR*, abs/1812.00564, 2018.

[16] Guanzhong Chen, Zhenghan Qin, Mingxin Yang, Yajie Zhou, Tao Fan, Tianyu Du, and Zenglin Xu. Unveiling the vulnerability of private fine-tuning in split-based frameworks for large language models: A bidirectionally enhanced attack. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, CCS '24, page 2904–2918. ACM, December 2024.

[17] Xicong Shen, Yang Liu, Yi Liu, Peiran Wang, Huiqi Liu, Jue Hong, Bing Duan, Zirui Huang, Yunlong Mao, Ye Wu, and Sheng Zhong. Sap: privacy-preserving fine-tuning on language models with split-and-privatize framework. In *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence*, IJCAI '25, 2025.

[18] Yae Jee Cho, Luyang Liu, Zheng Xu, Aldi Fahrezi, and Gauri Joshi. Heterogeneous LoRA for federated fine-tuning of on-device foundation models. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen, editors, *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 12903–12913, Miami, Florida, USA, November 2024. Association for Computational Linguistics.

[19] Youbang Sun, Zitao Li, Yaliang Li, and Bolin Ding. Improving loRA in privacy-preserving federated learning. In *The Twelfth International Conference on Learning Representations*, 2024.

[20] Hariharan Ramesh and Jyotikrishna Dass. Florist: Singular value thresholding for efficient and accurate federated fine-tuning of large language models. In A. Chowdhery and Z. Jia, editors, *Proceedings of Machine Learning and Systems*, volume 8, pages 1–21. MLSys, 2026.

[21] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. 2019.

[22] Jekaterina Novikova, Ondřej Dušek, and Verena Rieser. The E2E dataset: New challenges for end-to-end generation. In Kristiina Jokinen, Manfred Stede, David DeVault, and Annie Louis, editors, *Proceedings of the 18th Annual SIGdial Meeting on Discourse and Dialogue*, pages 201–206, Saarbrücken, Germany, August 2017. Association for Computational Linguistics.

[23] Meta AI. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[24] Karl Cobbe et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

[25] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. GLUE: A multi-task benchmark and analysis platform for natural language understanding. In Tal Linzen, Grzegorz Chrupała, and Afra Alishahi, editors, *Proceedings of the 2018 EMNLP Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*, pages 353–355, Brussels, Belgium, November 2018. Association for Computational Linguistics.

[26] Sunny Sanyal, Ravid Shwartz-Ziv, Alexandros G. Dimakis, and Sujay Sanghavi. When attention collapses: How degenerate layers in llms enable smaller, stronger models. *Trans. Mach. Learn. Res.*, 2026, 2024.

[27] Haoran Zhang, Dongjun Kim, Seohyeon Cha, and Haris Vikalo. FedRot-LoRA: Mitigating rotational misalignment in federated LoRA. *arXiv preprint arXiv:2602.23638*, 2026.

A Extended Experimental Details

Table 5: **Overview of experimental configurations.** All setups use non-IID data partitioned via Dirichlet (α=0.5) and exhaust every cut-layer position (or a uniform stride for LLaMA).

Model	Benchmark	Clients	Sampled	Rounds	LoRA rank
GPT-2 S/M/L	E2E NLG	3	3	100	$\{4, 8, 16\}$ (heter.)
GPT-2 S/M/L	E2E NLG	3	3	100	16 (homo.)
GPT-2 Small	GLUE (MNLI, QQP)	100	10	200	16 (homo.)
Llama-3-8B	GSM8K	30	3	50	16 (homo.)

A.1 GPT-2 on E2E NLG

Dataset. The E2E NLG benchmark [22] is a conditional generation dataset in the restaurant domain containing approximately 42K training, 4.6K validation, and 4.6K test samples. Each sample consists of a structured meaning representation and one or more reference utterances. We evaluate using five automatic metrics: BLEU, NIST, METEOR, ROUGE-L, and CIDEr.

Models and training. We evaluate GPT-2 Small (12 layers, 768 hidden, 124M parameters), Medium (24 layers, 1024 hidden, 355M), and Large (36 layers, 1280 hidden, 774M) [21]. All models are fine-tuned with batch size 8, learning rate 2×10^{-4}, and maximum sequence length 512, for 1 epoch (GPT-2 Small) or 2 epochs (Medium and Large). LoRA adapters ($\Delta\mathbf{W} = \mathbf{BA}$, $\mathbf{B} \in \mathbb{R}^{d\times r}$, $\mathbf{A} \in \mathbb{R}^{r\times d}$) are applied to the query and value projection matrices of every attention block.

Federated setup — heterogeneous (primary). N=3 clients, all participating every round. Client datasets are partitioned non-IID via Dirichlet (α=0.5), yielding skewed label distributions. LoRA ranks are assigned heterogeneously: $r_i \in \{4, 8, 16\}$ uniformly at random per client. Client adapters are aggregated every I=100 local optimization steps. Methods requiring aligned LoRA factors apply zero-padding rank alignment following HetLoRA [18].

Cut-layer sweep. The cut layer ℓ_c is swept exhaustively over $\{1, \ldots, L-1\}$ for each model and aggregation method, yielding $4 \times (L-1)$ independent SFF runs per model (48 for Small, 92 for Medium, 140 for Large). Under rank heterogeneity, all methods except STACK produce client-specific global adapters, requiring per-client evaluation and averaging, further multiplying experimental cost.

A.2 GPT-2 Small on GLUE

Dataset. We evaluate on two GLUE tasks: MNLI (Multi-Genre Natural Language Inference, 393K train / 9.8K val samples) and QQP (Quora Question Pairs, 364K train / 40K val samples). Both are cast as classification tasks.

Model and training. GPT-2 Small (12 layers, 124M) is fine-tuned with batch size 8, learning rate 2×10^{-4}, and maximum sequence length 512. LoRA adapters (r=16) are applied to query and value projections of every attention block.

Federated setup. 100 total clients, 10 sampled per round, for 200 communication rounds. Client datasets are partitioned non-IID via Dirichlet (α=0.5). All clients use homogeneous rank r=16. Client adapters are aggregated every I=100 local steps. This large-client configuration tests the scalability of the Depth-Performance Dilemma and aggregation sensitivity beyond the 3-client primary setup.

Cut-layer sweep. Same exhaustive sweep as the E2E experiments: $\ell_c \in \{1, \ldots, 11\}$, giving $4 \times 11 = 44$ independent SFF runs per GLUE task.

Results. The Depth-Performance Dilemma holds on both GLUE tasks: classification accuracy degrades consistently with cut-layer depth across all four aggregation methods, with the same aggregation ordering (SVD and STACK outperforming AVERAGE and FREEZE at deep cuts) observed in the E2E primary experiments. The 100-client setup amplifies aggregation-method differences at

depth relative to the 3-client setup, consistent with the $O(\sqrt{K})$ cross-task interference scaling of STACK and the $O(1/\sqrt{K})$ signal dilution of AVERAGE. Table 6 report accuracies at shallow and deep cut for MNLI and QQP respectively.

Table 6: GPT-2 Small results on GLUE benchmark (QQP and MNLI) with 100 clients (10 sampled per round) under homogeneous LoRA rank ($r = 16$). Shallow (resp. deep) values are obtained by averaging over the first (resp. last) 25% of possible cut layers. Best results per task and regime in **bold**; second-best underlined.

Task	Method	Shallow (%)	Deep (%)
QQP	AVERAGE	86.35	<u>83.60</u>
	FREEZE	84.66	79.34
	STACK	<u>84.24</u>	80.41
	SVD	**86.26**	**83.63**
MNLI	AVERAGE	<u>77.44</u>	<u>73.37</u>
	FREEZE	73.87	63.37
	STACK	73.50	66.51
	SVD	**77.55**	**73.39**

A.3 Llama-3-8B-Instruct on GSM8K

Dataset. GSM8K [24] is a benchmark of 8.5K grade-school mathematics word problems (7.5K train / 1K test) requiring multi-step arithmetic reasoning. We report exact-match accuracy on the test set.

Model and training. Llama-3-8B-Instruct [23] (32 Transformer layers, 4096 hidden dimension, 8B parameters) is fine-tuned with batch size 1, learning rate 1×10^{-4}, maximum sequence length 1024, and 1 local epoch per communication round. LoRA adapters (r=16) are applied to the key and value projection matrices of every attention block.

Federated setup. 30 total clients, 3 sampled per round, for 50 communication rounds. Client datasets are partitioned non-IID via Dirichlet (α=0.5). All clients use homogeneous rank r=16.

Cut-layer sweep. To manage the computational cost of exhaustive sweeping on a 32-layer, 8B model, cut layers are evaluated at stride 3: $\ell_c \in \{1, 4, 7, 10, 13, 16, 19, 22, 25, 28, 31\}$, giving $4 \times 11 = 44$ independent SFF runs. This stride is fine enough to capture the characteristic depth threshold at which performance acceleration occurs (observed around $\ell_c \approx 20$ for a 32-layer model, consistent with the attention collapse profile at roughly 60% depth), while remaining computationally tractable.

Results. The Depth-Performance Dilemma holds at 8B scale on mathematical reasoning: GSM8K accuracy degrades consistently with cut-layer depth across all four aggregation methods, with the same aggregation ordering observed in the GPT-2 experiments. The attention collapse threshold, measured by effective rank of attention matrices, shifts to $\ell \approx 20$ for LLaMA, consistent with the $\approx$60% depth pattern observed across the GPT-2 family. These results confirm that the dilemma is not an artifact of GPT-2's architecture or the E2E NLG task, but a general property of Transformer-based SFF. Figure 4 reports accuracy vs. cut-layer depth for all four aggregation methods.

Hardware setup and cost. All experiments are run on an HPC cluster equipped with NVIDIA H200 GPUs. Each individual SFF run for LLaMA, one cut-layer configuration under one aggregation method for 50 communication rounds, requires approximately 9 hours of wall-clock time. The full LLaMA sweep ($4 \times 11 = 44$ configurations) therefore represents roughly 396 GPU-hours of computation, underscoring the practical cost of exhaustive depth evaluation at 8B scale and motivating the stride-3 cut-layer sampling described above.

B Pseudocode

Algorithm 1: Split-Federated Fine-tuning (SFF); Aligned with Figure 2

Input: Pretrained LLM with L Transformer blocks $\{T_l\}_{l=1}^{L}$, cut layer l_c, K clients with private datasets $\{\mathcal{D}_k\}_{k=1}^{K}$, client LoRA ranks $\{r_k\}_{k=1}^{K}$, aggregation interval I steps, learning rates γ_C (client), γ_S (server)

Output: Fine-tuned model

Partition: Client k holds blocks $T_1, \ldots, T_{l_c}$ with LoRA adapters ($B_k \in \mathbb{R}^{d \times r_k}, A_k \in \mathbb{R}^{r_k \times d}$);
Main Server holds blocks $T_{l_c+1}, \ldots, T_L$ with shared parameters Θ_S;
Initialize: Global LoRA adapters $(B^{(g)}, A^{(g)})$;
for *each global round* $t = 1, 2, \ldots$ **do**
  Each client k downloads global adapters: $(B_k, A_k) \leftarrow (B_k^{(g)}, A_k^{(g)})$;
  /* Step ① */
  for *each local step* $s = 1, \ldots, I$ **do**
    /* **Step I:** Client-side Forward Pass (parallel) */
    for *each client* $k = 1, \ldots, K$ ***in parallel*** **do**
      Sample minibatch $(\mathbf{x}_k, \mathbf{y}_k) \sim \mathcal{D}_k$;
      $\mathbf{h}_{l_c,k} \leftarrow T_{l_c} \circ \cdots \circ T_1(\text{Embed}(\mathbf{x}_k))$;
      /* Step ②, using local LoRA (B_k, A_k) */
      Send smashed activations $\mathbf{h}_{l_c,k}$ to Main Server;
      /* Step ③ */
    /* **Step II:** Main Server Forward + Backward (sequential) */
    for *each client* $k = 1, \ldots, K$ ***sequentially*** **do**
      $\hat{\mathbf{y}}_k \leftarrow T_L \circ \cdots \circ T_{l_c+1}(\mathbf{h}_{l_c,k})$;
      /* Step ④ */
      Compute loss $\mathcal{L}_k = \mathcal{L}(\hat{\mathbf{y}}_k, \mathbf{y}_k)$;
      /* Step ⑤ */
      Backpropagate through Θ_S: compute $\nabla_{\Theta_S} \mathcal{L}_k$;
      /* Step ⑥ */
      Update: $\Theta_S \leftarrow \Theta_S - \gamma_S \nabla_{\Theta_S} \mathcal{L}_k$;
      /* Step ⑨ */
      Send gradient $\nabla_{\mathbf{h}_{l_c,k}} \mathcal{L}_k$ to client k;
      /* Step ⑦ */
    /* **Step III:** Client-side Backward Pass (parallel) */
    for *each client* $k = 1, \ldots, K$ ***in parallel*** **do**
      $(B_k, A_k) \leftarrow (B_k, A_k) - \gamma_C \nabla_{(B_k, A_k)} \mathcal{L}_k$;
      /* Step ⑧ */
  /* **Step IV:** Federated Aggregation on Fed Server (every I steps) */
  Clients upload local LoRA (B_k, A_k) to Fed Server;
  /* Step ⑫ */
  /* Any aggregation method can be used: Avg, Freeze, Stack, SVD */
  $(B_k^{(g)}, A_k^{(g)})_{k=1}^{K} \leftarrow \textsc{SpectralLoRA}\big(\{(B_k, A_k, r_k)\}_{k=1}^{K}\big)$;
  /* Step ⑬, Alg. 2 */

Algorithm 2: SPECTRALLORA: SVD-based Federated LoRA Aggregation (FlexLoRA)

Input: Local LoRA adapters $\{(B_k \in \mathbb{R}^{d\times r_k}, A_k \in \mathbb{R}^{r_k\times d})\}_{k=1}^K$, client weights $\alpha_k = n_k/N$ where $n_k = |\mathcal{D}_k|$, $N = \sum_k n_k$

Output: Client-specific global adapters $\{(B_k^{(g)}, A_k^{(g)})\}_{k=1}^K$

/* **Step 1:** SVD of global weight update */

Compute local updates: $\Delta W_k = B_k A_k \in \mathbb{R}^{d\times d}$ for each client k;

Aggregate: $\Delta W_{\text{agg}} = \sum_{k=1}^K \alpha_k \Delta W_k$;

Perform SVD: $\Delta W_{\text{agg}} = U\Sigma V^T$, where $U \in \mathbb{R}^{d\times d}$, $\Sigma = \text{diag}(\sigma_1, \ldots, \sigma_d)$, $V \in \mathbb{R}^{d\times d}$;

/* **Step 2:** Spectral filtering: truncate to client rank */

for *each client* $k = 1, \ldots, K$ **do**

  $B_k^{(g)} = U_{[:,:r_k]}\, \Sigma_{[:r_k,:r_k]}$;

  /* Top-r_k left singular vectors $\times$ singular values */

  $A_k^{(g)} = V^T_{[:r_k,:]}$;

  /* Top-r_k right singular vectors */

return $\{(B_k^{(g)}, A_k^{(g)})\}_{k=1}^K$

C Proof of Theorem C.1 (Throughput Scaling)

Theorem C.1: Throughput Scaling

Consider a Split-Federated system with K clients sampled to participate in a round and a main server, and let the computational cost of the Transformer blocks be linear with depth. In the server-bound regime $d < d^\star$ (Eq. (7)), as the partition depth ratio $d \to d^\star$ the system throughput $\mathcal{T}$ scales hyperbolically:

$$\mathcal{T}(d) = \Theta\left(\frac{1}{1-d}\right). \tag{1}$$

Example: Moving the client partition depth ratio from $d = 0.5$ to $d = 0.9$ yields a theoretical $5\times$ throughput increase, bounded only by the server's sequential processing capacity. (Proof in Appendix C).

Proof. Let W denote the total computational work (measured in FLOPs) required for a forward and backward pass on a single data batch for the full L-layer model. Since a Transformer model is composed of identical blocks (uniform hidden dimension d_{model} and sequence length S), the computational cost is linearly proportional to the model depth. For a partition depth ratio $d \in [0, 1)$, we define the work split as:

$$W_{\text{client}} = d \cdot W \tag{2}$$

$$W_{\text{server}} = (1-d) \cdot W \tag{3}$$

The system operates in synchronized rounds. The total time to complete one training round, T_{round}, is determined by the bottleneck between the parallel client execution phase and the sequential server execution phase. Let ρ_c and ρ_s be the computational rates (FLOPs/sec) of a single client device and the central server, respectively.

1. **Client Phase (Parallel):** All K clients compute their lower-model segments simultaneously.

$$T_{\text{client}} = \frac{W_{\text{client}}}{\rho_c} = \frac{d \cdot W}{\rho_c} \tag{4}$$

2. **Server Phase (Sequential):** The server must process the activations and gradients for all K clients.

$$T_{\text{server}} = \frac{K \cdot W_{\text{server}}}{\rho_s} = \frac{K \cdot (1-d) \cdot W}{\rho_s} \tag{5}$$

Table 7: **Throughput-optimal cut depth** $d^\star = K/(K + \rho_s/\rho_c)$ as a function of the participating clients per round K and the server-to-client compute ratio ρ_s/ρ_c (ρ_s/ρ_c=1: equally capable server and clients). The server stays the bottleneck for all $d < d^\star$; the Depth-Performance Dilemma holds whenever $d^\star$ reaches the attention-collapse onset ($\approx$ 50–60% depth).

K	$d^\star$ (ρ_s/ρ_c=10)	$d^\star$ (ρ_s/ρ_c=2)	$d^\star$ (ρ_s/ρ_c=1)
3	0.23	0.60	0.75
10	0.50	0.83	0.91
30	0.75	0.94	0.97
100	0.91	0.98	0.99

The system throughput $\mathcal{T}(d)$, defined as the number of client updates processed per second, is given by:

$$\mathcal{T}(d) = \frac{K}{T_{\text{round}}} = \frac{K}{\max(T_{\text{client}}, T_{\text{server}})} \tag{6}$$

The two phases scale oppositely in d: $T_{\text{client}} = dW/\rho_c$ grows with depth while $T_{\text{server}} = K(1 - d)W/\rho_s$ shrinks. The server is therefore the bottleneck ($T_{\text{server}} > T_{\text{client}}$) precisely when

$$\frac{K(1-d)}{\rho_s} > \frac{d}{\rho_c} \iff d < d^\star := \frac{K}{K + \rho_s/\rho_c}, \tag{7}$$

where $d^\star$ is the *throughput-optimal cut depth*, the crossover below which server-side compute dominates the round time. Within this server-bound regime, throughput is governed by the sequential server phase:

$$\mathcal{T}(d) \approx \frac{K}{T_{\text{server}}} = \frac{K}{\frac{K(1-d)W}{\rho_s}} = \frac{\rho_s}{W} \cdot \frac{1}{1-d}. \tag{8}$$

Since ρ_s/W is constant in d, we conclude $\mathcal{T}(d) = \Theta(1/(1-d))$, diverging hyperbolically as $d \to d^\star$. For $d > d^\star$ the client phase dominates and throughput instead *decreases* in d; the throughput incentive for deep cuts is thus confined to the server-bound regime, quantified next. □

Operating regime and the achievable cut range. The scaling $\Theta(1/(1 - d))$ governs the server-bound regime $d < d^\star$, so whether throughput rises across the *entire* achievable cut range depends on where $d^\star$ sits relative to the deepest valid cut. A valid split retains at least one server layer, so the maximum cut depth is $(L-1)/L$: $11/12$=0.91, $15/16$=0.94, and $35/36$=0.97 for GPT-2 Small, Medium, and Large. Table 7 reports $d^\star = K/(K+\rho_s/\rho_c)$ across participation levels K and compute ratios ρ_s/ρ_c. When $d^\star$ meets or exceeds $(L-1)/L$, the server is the bottleneck at every cut and throughput increases monotonically with depth, as plotted in Figure 1; for smaller $d^\star$, throughput peaks at $d^\star$ and the deep-cut incentive instead rests on privacy (below).

The dilemma holds across the operating range. For the Depth-Performance Dilemma to exist, $d^\star$ must land within the attention-collapse regime, which begins at 50–60% depth for all models (Figure 8). This holds across realistic settings: at ρ_s/ρ_c=10, $K \geq 10$ already gives $d^\star \geq 0.5$ (our 100-client GLUE setup samples K=10/round); at ρ_s/ρ_c=2, even K=3 gives $d^\star$=0.60; and at ρ_s/ρ_c=1 (server and clients equally capable), our E2E/GSM8K setup (K=3, $d^\star$=0.75) and GLUE setup (K=10, $d^\star$=0.91) both fall inside the collapse regime. Crucially, the dilemma does not rest on throughput alone: even where throughput favors shallow cuts (e.g., $d^\star$=0.23 at K=3, ρ_s/ρ_c=10) or the server is no longer the bottleneck ($d > d^\star$), privacy leakage still decays consistently with depth (Figure 3), independent of K, ρ, and execution model, so a privacy-driven deployment cuts deep at any K.

D Aggregation Noise Bounds

Assumption D.1: Heterogeneous Complexity

Assume, disjoint clients $k \in \{1 \dots K\}$ possess optimal updates ΔW_k^* associated with distinct local tasks. We define the Global Noise $\mathcal{E}$ as the deviation of the aggregated update ΔW_{agg} from the ideal multi-task manifold.

Theorem D.1: Aggregation Noise

Let $\Delta W_k = B_k A_k$ be the update from client k. The aggregation error $\mathcal{E}$ for each paradigm is lower-bounded by specific noise artifacts:

1. **AVERAGE ($\mathcal{E}_{avg}$): The Variance Noise.**
   Averaging forces a common rank r_{max}. Due to the nonlinearity of matrix multiplication ($\mathbb{E}[B]\mathbb{E}[A] \neq \mathbb{E}[BA]$) and the incoherence of client updates, the aggregate spectrum collapses:
   $$\mathcal{E}_{avg} \approx \underbrace{\text{Cov}(B, A)}_{\text{Algebraic Error}} + \underbrace{\frac{1}{\sqrt{K}} \|\Sigma_{noise}\|_F}_{\text{Spectrum Flattening}} \tag{9}$$
   The resulting update resembles high-rank "white noise", diluting specific task signals.
2. **STACK ($\mathcal{E}_{stack}$): The Interference Noise.** Aggregating via summation ($\Delta W_{stack} = \sum B_k A_k$) creates a "Cross-Task Interference" problem. For an input x belonging to Client k, the adapters of all other clients $j \neq k$ act as adversarial perturbations:
   $$\mathcal{E}_{stack} = \| \sum_{j \neq k} x B_j A_j \|_F \propto \sqrt{K-1} \cdot \sigma_{cross} \tag{10}$$
   In deep split networks ($K \gg 1$), this crosstalk variance overwhelms the local signal magnitude.
3. **FREEZE ($\mathcal{E}_{freeze}$): The Bias Noise.**
   Fixing a shared projection A_{fixed} constraints the global update $\Delta W_{freeze} = \bar{B} A_{fixed}$ to a fixed subspace $\mathcal{S}_A$. The error is the unlearnable residual energy of the ideal global update:
   $$\mathcal{E}_{freeze} = \|\Delta W_{ideal}^* - \text{proj}_{\mathcal{S}_A}(\Delta W_{ideal}^*)\|_F \tag{11}$$
   This constitutes an irreducible **Approximation Bias** for any task features lying in the orthogonal complement $\mathcal{S}_A^{\perp}$.

D.1 Proof of Theorem D.1 (The Aggregation Noise)

In this section, we rigorously derive the error bounds for the three dominant LoRA aggregation paradigms: AVERAGE (FedIT [10]), STACK (FLoRA [9]), and FREEZE (FFA-LoRA [19]).

D.1.1 Setup and Definitions

Let there be K clients. Each client k possesses an optimal update ΔW_k^* required to solve their local task. The **Ideal Global Update** is the average of these optimal directions (assuming a uniform objective):

$$\Delta W_{ideal}^* = \frac{1}{K} \sum_{k=1}^{K} \Delta W_k^* \tag{12}$$

The **Aggregation Error** $\mathcal{E}$ is the Frobenius norm of the difference between the actual aggregated update and this ideal update.

D.1.2 Part 1: AVERAGE (FedIT [10]) (The Variance Noise)

Mechanism: The server computes $\bar{B} = \frac{1}{K}\sum B_k$ and $\bar{A} = \frac{1}{K}\sum A_k$. The aggregated update is $\Delta W_{avg} = \bar{B}\bar{A}$.

Proof: Expanding the product of sums:

$$\Delta W_{avg} = \left(\frac{1}{K}\sum_i B_i\right)\left(\frac{1}{K}\sum_j A_j\right) \tag{13}$$

$$= \frac{1}{K^2}\sum_i B_i A_i + \frac{1}{K^2}\sum_{i\neq j} B_i A_j \tag{14}$$

$$= \frac{1}{K}\Delta W^*_{ideal} + \underbrace{\frac{1}{K^2}\sum_{i\neq j} B_i A_j}_{\text{Algebraic Noise}} \tag{15}$$

where the first equality uses $\frac{1}{K^2}\sum_i B_i A_i = \frac{1}{K}\cdot\frac{1}{K}\sum_i \Delta W_i^* = \frac{1}{K}\Delta W^*_{ideal}$. The signal term therefore scales as $O(1/K)$ relative to each client's update magnitude.

Spectral Analysis (Justification of "Flattening"): We analyze the spectral norm of the algebraic noise term $N = \frac{1}{K^2}\sum_{i\neq j} B_i A_j$. Assuming the cross-terms $B_i A_j$ are independent isotropic variables with variance σ^2, the sum consists of $K(K-1)$ terms. By the universality of random matrices (Marchenko-Pastur law), the singular values of N converge to a bulk distribution bounded by:

$$\|N\|_2 \approx \frac{1}{K^2}\cdot\sqrt{K(K-1)}\cdot\sigma \approx \frac{\sigma}{\sqrt{K}} \tag{16}$$

Crucially, this noise energy is not concentrated in a specific direction (like the signal) but is spread across all dimensions d. This transforms the spectral profile of the weight update:

- **Before Averaging:** ΔW_k is low-rank (spiky spectrum).
- **After Averaging:** ΔW_{avg} is high-rank (flat spectrum).

We term this phenomenon **Spectrum Flattening**, as the noise floor rises to $\propto 1/\sqrt{K}$, obscuring the singular values of the true signal. Thus, averaging effectively "flattens" the sharp, task-specific features of $B_k A_k$ into a high-rank, low-magnitude noise floor.

D.1.3 Part 2: STACK (FLoRA [9]) (The Interference Noise)

Mechanism: The server maintains the sum of all adapters. For an input x, the forward pass is $xW_{base} + \sum_{j=1}^{K} xB_j A_j$.

Mechanism: The server sums the updates $\Delta W_{stack} = \sum_k B_k A_k$. During inference, for an input x belonging to Client k, the model computes:

$$y = xW_{base} + \frac{1}{K}\sum_{j=1}^{K} xB_j A_j \tag{17}$$

Proof: The ideal activation for input $x^{(k)}$ is $x^{(k)}W_{base} + x^{(k)}B_k A_k$. The error is the **Interference Sum** from all other clients:

$$\mathcal{E}_{stack} = \left\| \frac{1}{K}\sum_{j\neq k} x^{(k)} B_j A_j \right\|_F \tag{18}$$

Modeling the interference terms $z_j = x^{(k)}B_j A_j$ as independent random vectors with variance σ^2 (relative to the subspace of task k), the total interference variance is:

$$\text{Var}(\mathcal{E}_{stack}) \approx (1 - 1/K)\sigma^2 \tag{19}$$

Thus, the Root Mean Square (RMS) error scales as:

$$\mathcal{E}_{stack} \propto \frac{\sigma}{\sqrt{K}} \tag{20}$$

Thus, the noise magnitude scales with $(\sqrt{K})^{-1}$. Signal-to-Noise Ratio (SNR) vanishes as $K \to \infty$ because the signal shrinks faster than the noise.

D.1.4 Part 3: FREEZE (FFA-LoRA [19]) (The Bias Noise)

Mechanism: A projection matrix A_{fixed} is frozen. Clients optimize B_k. The global model is aggregated as:

$$\Delta W_{freeze} = \left(\frac{1}{K} \sum_{k=1}^{K} B_k \right) A_{fixed} = \bar{B} A_{fixed} \tag{21}$$

Proof: Let $\mathcal{S}_A$ be the row space spanned by the fixed matrix A_{fixed} (rank r). By definition, the global update ΔW_{freeze} lies entirely within $\mathcal{S}_A$. However, the **Ideal Global Update** $\Delta W^*_{ideal} = \frac{1}{K} \sum \Delta W^*_k$ is a summation of K diverse task updates. If tasks are heterogeneous, their principal components span a union of subspaces with effective rank $R_{eff} \gg r$. The error is the projection loss of the ideal update onto the restricted subspace $\mathcal{S}_A$:

$$\mathcal{E}_{freeze} = \| \Delta W^*_{ideal} - \text{proj}_{\mathcal{S}_A}(\Delta W^*_{ideal}) \|_F \tag{22}$$

Substituting the sum:

$$\mathcal{E}_{freeze} = \left\| \frac{1}{K} \sum_{k=1}^{K} \underbrace{\left(\Delta W^*_k - B^{opt}_k A_{fixed} \right)}_{\text{Local Residuals } r_k} \right\|_F \tag{23}$$

Since the local residuals r_k (the parts of the tasks orthogonal to A_{fixed}) are unlikely to cancel out perfectly in a non-IID setting, this constitutes an irreducible **Approximation Bias**. The global model effectively "blind" to any task features that lie in the orthogonal complement $\mathcal{S}_A^{\perp}$.

□

E Analysis of Spectral Denoising

Theorem E.1: Spectral Denoising

Spectral Truncation improves the Signal-to-Noise Ratio (SNR) by projecting orthogonal noise out of the signal subspace.

Proof. Let the true signal be S (rank r) and the noise be N (rank d, resulting from incoherent aggregation). We assume a standard signal processing model where noise is isotropic (spherical): $N = \epsilon G$, where G has i.i.d. Gaussian entries and ϵ is the noise magnitude.

1. Energy Distribution: The total energy of the noise is $\mathbb{E}[\|N\|_F^2] = d^2\epsilon^2$. However, because the noise is high-rank (Theorem E.1), this energy is spread roughly equally across all d dimensions of the space. The energy contained within any r-dimensional subspace is proportional to the fraction of dimensions:

$$E_{projected} \approx \frac{r}{d} E_{total} = \frac{r}{d}(d^2\epsilon^2) = rd\epsilon^2$$

2. The Filtering Operation: Spectral LoRA projects the data onto the top-r singular vectors. Since S is rank-r, it lies almost entirely within this subspace (assuming signal $\gg$ noise). The retained noise is only the component of N that aligns with S. The retained noise energy is reduced by a factor of $\frac{r}{d}$.

3. SNR Improvement: Let $\text{SNR}_{in} = \frac{\|S\|_F^2}{\|N\|_F^2}$. The output SNR after SVD is:

$$\text{SNR}_{out} = \frac{\|S\|_F^2}{\|N_{projected}\|_F^2} \approx \frac{\|S\|_F^2}{\frac{r}{d}\|N\|_F^2} = \frac{d}{r}\text{SNR}_{in}$$

For a typical LLaMA model ($d = 4096$) and LoRA rank ($r = 16$), the SNR improvement is $\frac{4096}{16} = 256\times$. This theoretical gain explains why Spectral LoRA prevents the degradation of performance at deep cuts, where naive averaging fails. □

F Complexity Analysis

Table 8: **Computational complexity in Split-Federated Fine-Tuning (SFF).** Per-epoch local training cost is denoted by $\mathcal{T}(\cdot)$ and scales linearly with the number of trainable Transformer blocks. ℓ_c is the client-side cut depth (number of blocks executed on the client), and $L - \ell_c$ blocks are executed on the main server. m is the embedding size, n is the context length, $|D_k|$ is the number of local training samples, K is the number of clients, r_k is the LoRA rank on client k, $r = \sum_{k=1}^{K} r_k$, and r_s denotes the LoRA rank used for server-side adapters. Fed server costs correspond to aggregating *client-side* LoRA (replace ℓ_c by L if LoRA is applied to all layers).

METHOD	CLIENT	MAIN SERVER (SPLIT TAIL)	FED SERVER (AGGREGATION)
FULL FT	$\mathcal{O}(E \cdot \mathcal{T}(m, n, \lvert D_k\rvert; \ell_c))$	$\mathcal{O}(E \cdot \mathcal{T}(m, n, r_s, \lvert D_k\rvert; L - \ell_c))$	$\mathcal{O}(l_c K m n)$
AVERAGE	$\mathcal{O}(E \cdot \mathcal{T}(m, n, r_k, \lvert D_k\rvert; \ell_c))$	$\mathcal{O}(E \cdot \mathcal{T}(m, n, r_s, \lvert D_k\rvert; L - \ell_c))$	$\mathcal{O}(\ell_c(m+n)\, r)$
FREEZE	$\mathcal{O}(E \cdot \mathcal{T}(m, n, r_k, \lvert D_k\rvert; \ell_c))$	$\mathcal{O}(E \cdot \mathcal{T}(m, n, r_s, \lvert D_k\rvert; L - \ell_c))$	$\mathcal{O}(\ell_c\, n\, r)$
STACK	$\mathcal{O}(E \cdot \mathcal{T}(m, n, r_k, \lvert D_k\rvert; \ell_c)) + \mathcal{O}\big(\ell_c\, m\, n \sum_{k=1}^{K} r_k\big)$	$\mathcal{O}(E \cdot \mathcal{T}(m, n, r_s, \lvert D_k\rvert; L - \ell_c))$	NONE
SVD	$\mathcal{O}(E \cdot \mathcal{T}(m, n, r_k, \lvert D_k\rvert; \ell_c))$	$\mathcal{O}(E \cdot \mathcal{T}(m, n, r_s, \lvert D_k\rvert; L - \ell_c))$	$\mathcal{O}(\ell_c\, mn\, r) + \mathcal{O}(\ell_c \min(m, n)\, mn) + \mathcal{O}\big(\ell_c\, m\, r^2\big)$

Table 9: **Communication overhead per round in Split-Federated Fine-Tuning (SFF).** We decompose communication into (i) federated LoRA parameter exchange between clients and the Fed server, and (ii) bidirectional split-learning traffic between clients and the main server. I denotes the number of local client optimization steps per round, B is the batch size; each step incurs transmission of cut activations and gradients of size $\Theta(nm)$. **Observation:** Split-learning traffic scales with the full activation dimension $\Theta(Bnm)$ and typically dominates communication cost, exceeding LoRA parameter exchange with fed server by orders of magnitude for practical ranks.

METHOD	CLIENT→FED (UPLOAD LORA)	FED→CLIENT (DOWNLOAD LORA)	CLIENT↔MAIN (SPLIT TRAFFIC)
FULL FT	$\mathcal{O}(\ell_c K \cdot mn)$	$\mathcal{O}(\ell_c K \cdot mn)$	$\mathcal{O}(KIB \cdot nm)$
AVERAGE	$\mathcal{O}(\ell_c(m+n)\, r)$	$\mathcal{O}(\ell_c K(m+n)\, \max(r_k))$	$\mathcal{O}(KIB \cdot nm)$
FREEZE	$\mathcal{O}(\ell_c\, n\, r)$	$\mathcal{O}(\ell_c K\, n\, \max(r_k))$	$\mathcal{O}(KIB \cdot nm)$
STACK	$\mathcal{O}(\ell_c(m+n)\, r)$	$\mathcal{O}(\ell_c K(m+n)\, r)$	$\mathcal{O}(KIB \cdot nm)$
SVD	$\mathcal{O}(\ell_c(m+n)\, r)$	$\mathcal{O}(\ell_c(m+n)\, r)$	$\mathcal{O}(KIB \cdot nm)$

G Empirical Privacy Evaluation via Inversion Attacks

We quantify information leakage at each cut layer by training adversarial inversion models that attempt to reconstruct input tokens from the cut-layer hidden states exposed to the server.

Setup. We evaluate GPT-2 Small (12 layers, hidden size 768), Medium (24 layers, 1024), and Large (36 layers, 1280) under a split inference setting. Given an input sequence $x = (x_1, \ldots, x_T)$, the client computes the intermediate representation at cut layer ℓ_c:

$$\mathbf{z}^{(\ell_c)} = f_{\leq \ell_c}(x) \in \mathbb{R}^{T \times D}, \tag{24}$$

where $f_{\leq \ell_c}$ denotes the Transformer truncated at layer ℓ_c and D is the hidden dimension. These representations are treated as the observable interface accessible to an adversary, the sole information the server receives about the client's private input.

Inversion model. For each cut layer, we train a separate adversarial inversion model $g_\theta : \mathbb{R}^D \to \{1, \ldots, V\}$, implemented as a token-wise MLP with an input projection ($D \to H$), a ReLU non-linearity, and an output layer ($H \to V$), where V is the vocabulary size. Training data consists of aligned representation–token pairs: $\mathcal{D}_{\ell_c} = \{(\mathbf{z}_t^{(\ell_c)}, x_t)\}_{t=1}^N$, constructed by passing sequences through the frozen truncated model and removing padding tokens. The model is trained with cross-entropy loss:

$$\mathcal{L} = -\mathbb{E}_{(\mathbf{z},x)\sim\mathcal{D}_{\ell_c}} \log P_\theta(x \mid \mathbf{z}), \tag{25}$$

using AdamW for a fixed number of epochs. This setup assumes a strong adversary with access to representation–token pairs and knowledge of the data distribution, a worst-case leakage scenario.

Metrics. After training, the inversion model is evaluated on the same representation distribution. We measure the *Recovery Rate* (RR), defined as token reconstruction accuracy:

$$\mathrm{RR} = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}[\hat{x}_i = x_i], \quad \hat{x}_i = \arg\max_v P_\theta(v \mid \mathbf{z}_i). \tag{26}$$

We report *Empirical Privacy* $\mathrm{EP} = (1 - \mathrm{RR}) \times 100$, normalized to $[0, 100]$. A score of 0 indicates perfect token reconstruction by the attacker (no privacy); a score of 100 indicates complete failure of reconstruction (full privacy). This metric is computed independently for each cut layer, yielding the depth-vs-privacy curves reported in Figure 3.

Extension to Llama-3-8B. We repeat the inversion protocol on Llama-3-8B-Instruct (32 layers, hidden size 4096) using the same MLP attacker and worst-case white-box assumptions. Figure 9 shows the same qualitative trend observed for the GPT-2 family: the normalized privacy score increases with depth, rising steeply through the early layers, plateauing across the middle depths, and saturating at 100 at the deepest cut. This confirms that the depth–privacy relationship is not an artifact of the GPT-2 architecture but holds at 8B scale, consistent with the data-processing view that successive Transformer blocks can only reduce the mutual information $I(x; \mathbf{h}_{\ell_c})$ between the cut-layer representation and the private input.

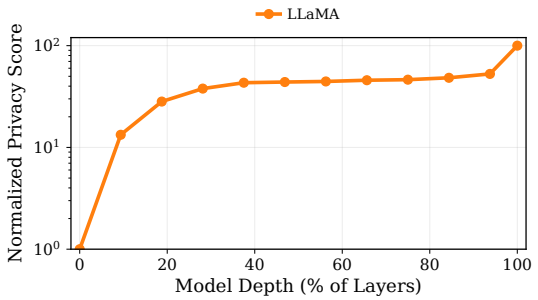
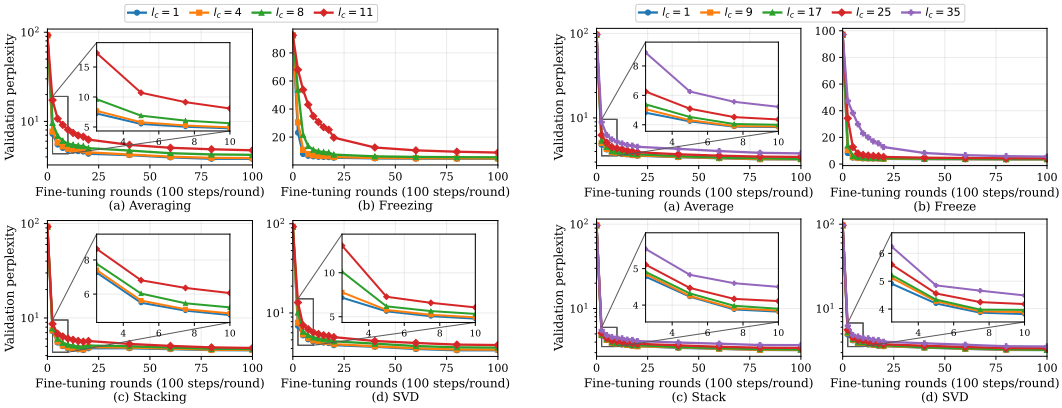


Figure 9: **Normalized Privacy Score vs. partition depth for Llama-3-8B-Instruct.** Score$=(1-\mathrm{RR})\times 100$ (log-scale y-axis), where RR is the token recovery rate of the adversarial MLP inversion model. As with the GPT-2 family (Figure 3), leakage decays with depth.

H Additional Experiments



(a) GPT-2 Small (b) GPT-2 Large

Figure 10: **Convergence of GPT-2 Small and Large in heterogeneous setting** Validation perplexity (PPL; lower is better) vs. rounds (100 steps/round) for cut depth l_c. Main panels use log-y (except Freezing in (b), linear-y); insets (when shown) zoom early training with linear-y. **Observation:** Deeper cuts slow convergence and worsen final PPL; Stacking and SVD are consistently more stable than Averaging and Freezing.

Table 10: Comparison of average E2E scores of GPT2-S, GPT2-M and GPT2-L across cutlayers for ranks 8 and 16 under homogenous settings. We report a unified E2E score computed as the arithmetic mean of per-metric min–max normalized scores across ROUGE-L, METEOR, BLEU, CIDEr, and NIST.

	Shallow Cut ($L/4$)				Middle Cut ($L/2$)				Deep Cut ($3L/4$)			
Model	**Avg**	**Frz**	**Stk**	**SVD**	**Avg**	**Frz**	**Stk**	**SVD**	**Avg**	**Frz**	**Stk**	**SVD**
GPT2-S ($r=8$)	57.4	53.3	57.2	**57.9**	57.9	**58.6**	53.1	58.0	**57.9**	56.7	53.5	56.1
GPT2-S ($r=16$)	**57.1**	53.9	56.5	56.0	56.0	56.4	53.5	**58.0**	56.6	**56.8**	53.0	52.8
GPT2-M ($r=8$)	**60.3**	56.2	58.5	59.8	59.8	**59.9**	56.3	57.7	**60.4**	59.7	56.6	59.2
GPT2-M ($r=16$)	58.4	55.9	58.5	**58.7**	**58.7**	57.6	56.0	57.4	**57.7**	57.5	56.5	57.6